

THE TAILWIND CATALOGUE

Not the lecture. The thing you keep open in the other window while you work.

WHAT THIS IS

The translation table, the three scales, the variants, and the syntax.

WHAT IT IS NOT

A list to memorise. Look things up here; do not learn it.

BETTER THAN THIS

`demo/03-translator/` answers by measuring.
Use it first.

BOX — SPACING, SIZE, BORDER

CSS YOU WROTE	UTILITY	NOTE
<code>padding: 1rem</code>	<code>p-4</code>	4 × the 0.25rem step
<code>padding-inline: 1.5rem</code>	<code>px-6</code>	<code>py-*</code> for block
<code>padding-top: .5rem</code>	<code>pt-2</code>	<code>pt</code> , <code>pr</code> , <code>pb</code> , <code>pl</code>
<code>margin: 0</code>	<code>m-0</code>	<code>-mt-4</code> is negative
<code>margin-inline: auto</code>	<code>mx-auto</code>	the centring one
<code>margin-block-start: auto</code>	<code>mt-auto</code>	pushes to the bottom of a flex column
<code>width: 100%</code>	<code>w-full</code>	<code>w-screen</code> , <code>w-fit</code> , <code>w-4</code>
<code>max-width: 36rem</code>	<code>max-w-xl</code>	named steps, not the spacing scale
<code>border: 1px solid</code>	<code>border</code>	<code>border-2</code> , <code>border-t</code>
<code>border-radius: .75rem</code>	<code>rounded-xl</code>	<code>rounded-full</code> for a pill
<code>box-sizing: border-box</code>	—	already the default. It is in the reset.

LAYOUT — FLEX AND GRID

CSS YOU WROTE	UTILITY	NOTE
<code>display: flex</code>	<code>flex</code>	<code>inline-flex</code> , <code>grid</code> , <code>block</code> , <code>hidden</code>
<code>flex-direction: column</code>	<code>flex-col</code>	<code>flex-row</code> is the default
<code>flex-wrap: wrap</code>	<code>flex-wrap</code>	grid never needs it
<code>flex: 1 1 0%</code>	<code>flex-1</code>	<code>flex-auto</code> , <code>flex-none</code>
<code>flex-basis: 0</code>	<code>basis-0</code>	equal shares ignoring content
<code>gap: 1rem</code>	<code>gap-4</code>	<code>gap-x-*</code> , <code>gap-y-*</code>
<code>grid-template-columns: repeat(3,1fr)</code>	<code>grid-cols-3</code>	up to <code>grid-cols-12</code>
<code>grid-column: span 2</code>	<code>col-span-2</code>	<code>col-start-*</code> , <code>col-end-*</code>
<code>grid-template-areas</code>	no utility	use the 12-column form, or brackets
<code>min-width: 0</code>	<code>min-w-0</code>	week 3's overflow fix, unchanged
<code>position: sticky</code>	<code>sticky</code>	with <code>top-0</code>

TYPE AND COLOUR

CSS	UTILITY
<code>font-size: .875rem</code>	<code>text-sm</code>
<code>font-size: 1.5rem</code>	<code>text-2xl</code>
<code>font-weight: 600</code>	<code>font-semibold</code>
<code>line-height: 1.25</code>	<code>leading-tight</code>
<code>text-align: center</code>	<code>text-center</code>
<code>text-decoration: none</code>	<code>no-underline</code>
<code>text-transform: uppercase</code>	<code>uppercase</code>
<code>font-variant-numeric</code>	<code>tabular-nums</code>

CSS	UTILITY
<code>color: ...</code>	<code>text-brand-500</code>
<code>background: ...</code>	<code>bg-surface</code>
<code>border-color: ...</code>	<code>border-edge</code>
<code>outline-color: ...</code>	<code>outline-brand-500</code>
<code>opacity on a colour</code>	<code>bg-white/15</code>
<code>opacity: .5</code>	<code>opacity-50</code>

text-* is both. text-sm is a size, text-slate-600 is a colour. The value decides.

THE THREE SCALES

SPACING — 0.25rem PER STEP

0 · 0.5 · 1 · 1.5 · 2 · 3 · 4 ·

Seeing a number in old CSS? **Divide by four.** 16px is 4.

TYPE — NAMED, NOT NUMBERED

xs · sm · base · lg · xl · 2xl ·

Each carries a line-height with it.
Override with `leading-*`.

COLOUR — 50 → 950

brand-100 · brand-300 ·
brand-500 · brand-700

Higher is darker. **Which steps are legal for text is a contrast question**, not a palette one.

All three come out of `@theme`. Add a name there and its whole family of utilities exists.

RESPONSIVE PREFIXES

PREFIX	GENERATES	ROUGHLY
(none)	no query at all	every width. Start here.
sm:	<code>@media (width >= 40rem)</code>	640px
md:	<code>@media (width >= 48rem)</code>	768px
lg:	<code>@media (width >= 64rem)</code>	1024px
xl:	<code>@media (width >= 80rem)</code>	1280px
2xl:	<code>@media (width >= 96rem)</code>	1536px
max-md:	<code>@media (width < 48rem)</code>	the escape hatch. Not a default.

THEY STACK, THEY DO NOT TAKE TURNS

At 1400px, `sm`, `md`, `lg` and `xl` all match. The last one that matches ends up on top. None of them fights.

THE BREAKPOINTS ARE IN `rem`

So a reader who enlarged their browser font gets them earlier — which is correct, and is why they are not written in pixels.

STATE VARIANTS — THE ONES YOU REACH FOR

VARIANT	GENERATES	NOTE
<code>hover:</code>	<code>@media (hover: hover) + :hover</code>	never sticks on a phone — free
<code>focus-visible:</code>	<code>:focus-visible</code>	the rubric line. Keyboard only
<code>focus:</code>	<code>:focus</code>	fires on mouse too. Usually you want the one above
<code>active:</code>	<code>:active</code>	only while the button is held down
<code>disabled:</code>	<code>:disabled</code>	a state, not a fifth variant
<code>dark:</code>	<code>@media (prefers-color-scheme: dark)</code>	the OS preference. No button
<code>motion-safe:</code>	<code>@media (prefers-reduced-motion: no-preference)</code>	put every transition behind it
<code>placeholder:</code>	<code>::placeholder</code>	pseudo-elements work too

THE TWO THAT LOOK LIKE THEY NEED JAVASCRIPT

VARIANT	GENERATES	USE IT FOR
<code>group-hover:</code>	<code>:is(:where(.group):hover *)</code>	parent hovered, child styled. Put <code>group</code> on the parent
<code>peer-checked:</code>	<code>:is(:where(.peer):checked ~ *)</code>	a control styling a later sibling. Put <code>peer</code> on the control

READ THEM OUT LOUD

“an element inside a hovered `.group`”, and “a later sibling of a checked `.peer`”. A combinator and a pseudo-class.

THE CONTROL MUST COME FIRST

`~` is the general sibling combinator. **CSS cannot look backwards**, so a `peer` after the thing it styles does nothing at all.

@theme — THE SYNTAX

```
1 <style type="text/tailwindcss"> <!-- the type attribute is what marks it -->
2   @theme {
3     /* a colour: gives you text-*, bg-*, border-*, ring-*, outline-* */
4     --color-brand-500: hsl(234 89% 52%);
5
6     /* the spacing multiplier: one number behind p-*, m-*, gap-*, w-*, h-* */
7     --spacing: 0.25rem;
8   }
9 </style>
```

IT IS ALSO A CUSTOM PROPERTY

`var(--color-brand-500)` works everywhere it worked in week 2. Nothing was replaced.

THE NAMING IS THE API

`--color-*` makes colour utilities.
`--text-*` makes sizes,
`--radius-*` makes corners. The prefix decides the family.

NO BUILD STEP

Which is also why the block is repeated on every page. In week 9 that stops being necessary.

ARBITRARY VALUES – SYNTAX, AND THE TEST

FORM	EXAMPLE	WHEN
arbitrary value	<code>w-[728px]</code>	a size somebody else decided
arbitrary property	<code>[mask-image:linear-gradient(...)]</code>	a property with no utility at all
underscores for spaces	<code>grid-cols-[repeat(auto-fit, minmax(11rem,1fr))]</code>	a pattern a utility cannot take
a theme value	<code>bg(--color-brand-500)</code>	rarely — the named class is better

“WOULD A SECOND ELEMENT WANT THIS EXACT VALUE?”

No → brackets are correct. Nobody chose it and nobody will reuse it.

YES

It is a decision. Put it in `@theme` and it gets a name.

ALREADY ON THE SCALE, OR ONE STEP OFF

This is the smell. `p-3` exists; `p-[13px]` is you losing.

WHAT YOU WILL SEE IN OLDER TUTORIALS

TAILWIND V3 — YOU WILL MEET THIS EVERYWHERE

```
<script src="https://cdn.tailwindcss.com"></script>
<script>
  tailwind.config = {
    theme: {
      extend: {
        colors: { brand: '#182df2' },
      },
    },
  };
</script>
```

V4 — WHAT WE USE

```
<script src="vendor/tailwind.js"></script>
<style type="text/tailwindcss">
  @theme {
    --color-brand-500: hsl(234 89% 52%);
  }
</style>
```

Same idea, and the class names barely moved. If a tutorial shows the left one, the classes still apply — only the config does not.

REPEATED MARKUP — THE THREE RUNGS

RUNG	WHAT IT IS	WHAT IT COSTS
copies	the same class list typed out N times	they drift within a fortnight, silently
a gallery page	defined once, copied from there	a discipline, so it can be broken. This is CA5
a function	one string, N calls	nothing. Week 9 , and it needs JavaScript
<code>@apply</code>	not a rung. Moves styles into a stylesheet	every cost of having a stylesheet, back

`@apply` 's one legitimate use: elements whose markup you do not control — prose links, `::selection`, a third-party widget.

THE FIVE, ONE LINE EACH

1. "IT'S INLINE STYLES"

A class carries conditions — hover, width, scheme. An attribute carries a value, at `1,0,0,0`.

2. MEMORISING THE NAMES

Know the CSS underneath. Ask [demos/03-translator/](#) for the rest.

3. @apply EVERYWHERE

Solves the wrong half. Six `class="btn"` is still six copies.

4. IGNORING THE SCALE

Rhythm is a property of the set. One off-scale box is invisible; three are not.

5. NO COMPONENT

The copy that loses a class looks perfect until somebody touches it.

WHERE TO LOOK, IN ORDER

1 `demos/03-translator/`

— paste CSS, get the class. It measures.

2 **Tailwind CSS IntelliSense**

— and it flags a class that does not exist.

3 `tailwindcss.com/docs`

— the search takes CSS property names.

4 **DevTools**

— the styles panel. It is ordinary CSS and always was.

**Starting at the docs is memorising.
Starting at the translator is translating.**